IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 5 September 2024, accepted 23 October 2024, date of publication 11 November 2024, date of current version 19 November 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3495703

RESEARCH ARTICLE

NAPE: Numbering as a Position Encoding in Graphs

OLAYINKA AJAYI[1], HONGKAI WEN[2], AND TANAYA GUHA[3]
[1]Mathematics Institute, University of Warwick, CV4 7AL Coventry, U.K.
[2]Department of Computer Science, University of Warwick, CV4 7AL Coventry, U.K.
[3]School of Computing Science, University of Glasgow, G12 8QQ Glasgow, U.K.

Corresponding author: Olayinka Ajayi (olayinka.ajayi@warwick.ac.uk)

This work was supported by the Engineering and Physical Sciences Research Council (EPSRC) through the Mathematics for Real-World Systems Centre for Doctoral Training (MathSys II).

ABSTRACT Deep learning has been instrumental in feature extraction from various data types, such as images and sequences, which inherently possess oriented structures. However, graph data, which delineate relationships between entities, lack such structures, posing challenges for embedding nodes uniquely in high-dimensional spaces. This paper introduces a novel position encoding method, Numbering as a Position Encoding (NAPE), which utilizes precomputed random walks and Hamming distances to approximate node orderings in graphs. NAPE assigns unique integers to nodes based on their local neighbourhoods, facilitating the generation of sinusoidal vectors that uniquely encode node positions. This method mitigates the computational expenses and non-uniqueness issues associated with eigenvector-based approaches, ensuring distinct node embeddings for various graph-based applications, including human action recognition. Our approach, scalable with computational complexity $\mathcal{O}(|V|^2)$, demonstrates improved efficiency and effectiveness in embedding nodes uniquely. Notably, NAPE improves the accuracy of action recognition models applied to human skeletal graphs. The versatility of NAPE is highlighted by its ability to generalize across different models, making it a robust solution for training action recognition systems with large parameter sizes.

INDEX TERMS Action recognition, computational efficiency, eigenvectors, graphs, human skeletal graphs, multi-graphs, NAPE, position encoding.

I. INTRODUCTION

Deep learning has proved useful in modern-day research due to its ability to extract features from raw data, without any handcrafted feature extraction techniques as seen in traditional machine learning. Data such as images, videos and sound used in deep learning come with inherent oriented structures like grids, mesh and sequences. This makes embedding such data to a higher dimensional Euclidean space possible without much concern about their position in space.

Graphs on the other hand, which establish the relationship between objects, do not have an oriented structure that allows each node in the graph to be embedded without the possible conflict of two or more nodes coinciding in the embedding space, especially when the local neighbourhood of such nodes are similar. This becomes a problem as nodes will not be distinguishable in the embedding space, which will affect retrieval systems using such graph-based algorithms.

Introducing additional information to the graph algorithm about each node's position in the graph would ensure that the embedding of these nodes is unique. The notion of position encodings was introduced to natural language processing (NLP) in [1] to account for the order of words in a sentence. This was necessary due to the transformer architecture not accounting for the order of the words in the sentence. Unlike sentences which form an ordered sequence, graphs do not come with an oriented ordered structure. Most literatures [2], [3], and [4] have sought to generate an equivalent of the transformer position encoding for graphs.

The transformer position encoding is closely related to the eigenvectors of the Laplacian of a path/sequential graph (a string of nodes) [3]. The authors in [2] used

The associate editor coordinating the review of this manuscript and approving it for publication was Walter Didimo.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

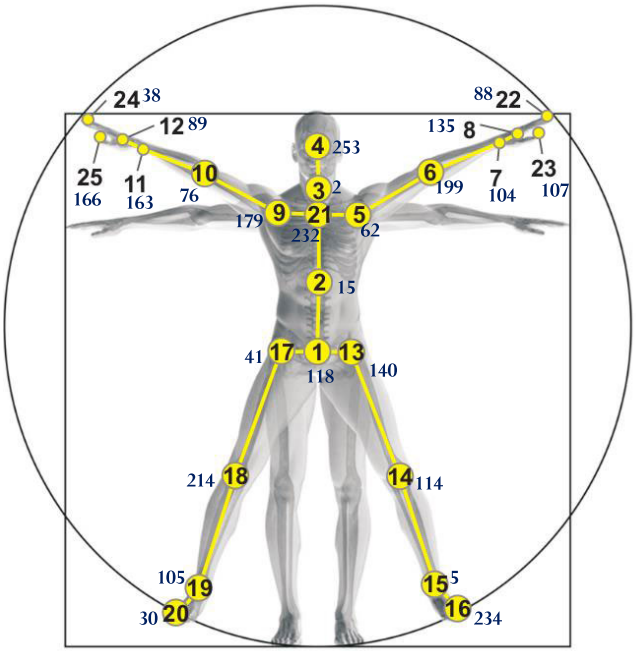


FIGURE 1. The figure shows a human skeletal graph with the nodes numbered/ordered canonically from 1 to 25. The figure also shows another numbering for each node using the NAPE algorithm. The NAPE ordering is used with the sinusoidal vector equation as position encodings for the skeletal graph in human action recognition models.

the eigenvectors of graph Laplacian as position encodings for non-sequential graphs. Their results show that this improves the classification accuracy of some benchmark datasets. Using eigenvectors as position encodings introduced a problem of non-uniqueness of the position encodings. The position encodings are used to identify the unique location of a node in a graph, but using eigenvectors generally defeats this uniqueness criterion since for any given eigenvector $\mathbf{x}$, $-\mathbf{x}$ is also an eigenvector. This is considered in the literature as sign flip invariance [4]. Also, due to the algebraic multiplicity of eigenvalues, we may not be able to generate a unique position encoding for each node in the graph.

Both [2] and [4] require that the eigenvector of the graph Laplacian be computed which is computationally expensive for very large graphs $\mathcal{O}(|V|^3)$, where $|V|$ is the number of nodes in the graph. According to the spectral graph theory [5], the eigenvectors define clusters of the nodes in a graph. So using eigenvectors as position encodings is equivalent to localizing a node within a cluster in the graph. Using precomputed random walks, we can identify the clusters in the graph without having to compute eigenvectors. Also using the clusters identified in the random walks it is possible to learn an ordering of the graph nodes (localizing each node within a cluster), such that one can adopt the sinusoidal vector approach (which does not compute eigenvectors) used in [1] for position encodings.

We proposed a novel position encoding algorithm that

- approximates the ordering of nodes in a graph using precomputed random walks and the Hamming distance of their binary representations;
- our method has a computational cost of $\mathcal{O}(|V|^2)$; and
- the learnt position encodings are suitable for any downstream task.

II. RELATED WORK

A. POSITION ENCODING

Position encodings have been embraced for sequential data since their use in [1]. They are useful for distinguishing similar representations that lie in different positions in the data. As relevant as they are in sequential data, they are also relevant in graph data. Position encoding is applied to graph data to distinguish the embedding of nodes that share similar local structures, but are in different positions in the graph [6]. Adding position encoding to a GNN ensures that the similar nodes would have different embeddings [6].

1) RELATIVE POSITION ENCODING

This approach encodes the relative positions of nodes within a specific neighbourhood. Reference [7] proposed the Positional Encoder Graph Neural Network (PE-GNN) for geographic data, which uses a learnable module to encode node coordinates, thereby capturing spatial context and improving performance on tasks like traffic prediction. Reference [8] expanded the receptive field of GNNs with attention mechanisms by adding new edges and nodes to the graph, along with positional encodings as features. However, these methods may not scale well for very large graphs due to the increased computational cost. You et al. [6] introduced P-GNNs, which incorporate relative positional information by using random walks to compute positional encodings, thereby capturing the relative distances between nodes and enhancing the model's understanding of graph structure.

2) SPECTRAL POSITIONAL ENCODING

This line of work uses inherent spectral graph properties for encoding positions. The Laplacian Eigenvector approach [2] computes the eigenvectors of the normalized Laplacian matrix to generate position encodings for graph nodes. These encodings capture the connectivity structure of the graph and have been shown to improve performance in tasks like node classification [2], [3]. However, the inherent sign ambiguity of the eigenvectors presents a challenge. To address this, the authors propose randomly flipping the eigenvectors' signs during training to ensure sign-invariant position encodings. Huang et al. [9] introduced Stable and Expressive Positional Encodings (SPE), an architecture that uses eigenvalues to softly partition eigenspaces. SPE is proven to be stable and universally expressive for basis invariant functions while respecting all symmetries of eigenvectors.

3) NODE EMBEDDING-BASED POSITIONAL ENCODING

This technique uses the node embeddings for a graph in place of position encodings, particularly for graphs having node features [10], [11], [12]. Reference [10] utilized wavelet

diffusion patterns in graphs, which are spectral filters, to encode positional information and structural roles of nodes. This approach captures both local and global structural features by propagating wavelets through the graph and using the resulting diffusion patterns as node embeddings.

4) DISTANCE-BASED ENCODINGS

Shortest path encoding uses the shortest path distance between nodes as a feature to provide relative positional information [13], [14]. While this method is effective for capturing distance-based relationships, it can become computationally intensive for large graphs. Adjacency power encoding raises the adjacency matrix to a power, effectively capturing information about node neighbourhoods up to a certain hop-count [15]. This method is computationally efficient and enhances the receptive field of GNNs, but it might lose fine-grained positional information as the hop-count increases.

5) LEARNABLE ENCODINGS

Recent work explores learning positional encodings jointly with the GNN model. Reference [16] proposes a novel architecture (LSPE) that separates structural and positional representations. LSPE utilizes learnable modules for both aspects, allowing the network to adapt the encodings to the specific task and data. To address the sign flip issue and the basis ambiguity problem associated with eigenvector-based methods [2], [4] developed neural networks, SignNet and BasisNet. These networks approximate continuous functions of the eigenvectors, ensuring invariance to sign changes under certain conditions. Reference [17] introduced positional encodings based on sheaf theory in graphs which help to break the locality constraint in traditional position encoding algorithms, by informing the nodes of their global positions in the graph. Learnable positional encodings are used in [18] which is a transformer-based action recognition algorithm to improve the embeddings of skeletal graphs and achieve state-of-the-art accuracy on the NTU-RGB+D datasets.

6) TRANSFORMER-BASED POSITION ENCODINGS

Reference [1] introduced sinusoidal position encodings to distinguish word embeddings within sentences, particularly benefiting their attention-based embedding technique over Recurrent Neural Networks (RNNs) like LSTM. This encoding method is modelled after a directed path graph, whose Laplacian eigenvectors form sinusoidal vectors, thus providing unique and informative encodings that capture the relative positions of words within a sentence. Reference [3] highlighted the importance of position encodings in graph structures. Their work integrates the concept of directionality in graph data, extending the utility of transformers from sequential to graph-based data structures.

III. PROPOSED METHOD

Our proposed algorithm builds on eigenvectors of directed path graphs used as positional encodings in the Transformer model [1]. The Transformer model used in NLP is applied directly to sentences, which can be modelled as a directed path graph whose nodes are already ordered. The ordering of these nodes (words) is passed as input to the positional encoding function.

For a simple graph $\mathcal{G}(V, E)$ whose nodes, V, have no ordering, our algorithm learns this ordering as a binary vector by approximating distances within clusters of nodes using the hamming distance. The clustering algorithm we use follows from the popular contrastive learning paradigm [19], [20], [21], [22] (see (2) and (3)). We have only replaced the distance metric, which was previously a dot product in past literature, with the hamming distance. Our approach to scaling our algorithm follows from the work of node2vec [11] where the authors used the negative sampling technique [23]. These existing techniques provide sufficient theoretical background for our work, as they are the foundation we have built upon.

In the introduction, we discussed how the eigenvectors define clusters within a graph. We further discussed the expensive computational cost of $\mathcal{O}(|V|^3)$ required to obtain these eigenvectors. The memory requirements to compute and store the eigenvectors for very large graphs are additional drawbacks for the Laplacian eigenvector method. A central theme for the positional encoding approaches we covered in the related work section is their inability to scale well for very large graphs. We have designed our algorithm to capture clusters within a graph using binary vectors, and our approach scales well to large graphs, without facing constraints of memory or computational resources like other methods discussed.

A. NUMBERING AS A POSITION ENCODING

Given a graph $\mathcal{G}(V, E)$ with V as the set of nodes, E being the set of edges in the graph and $|V| = N$, our work introduces a novel approach to position encoding in a graph that aims to assign each node to a unique integer based on its local neighbours.

Numbering a sequence or grid is trivial since we can identify each element in increasing order as they appear ie. $1, 2, 3, \ldots, N$ or orient each node of a grid in space. This is not the case for a graph $\mathcal{G}(V, E)$, as there are $N!$ possible ways to number each node in the graph. Just like a sequence, we desire a systematic way to number each node in $\mathcal{G}(V, E)$, such that the number assigned to a node captures its local neighbourhood.

Definition 1: Given $l > 0$ and $i, j \in V$. i and j are in the same local neighbourhood if $\text{dist}(i, j) \leq l$, where dist(.) is some distance metric. When $\text{dist}(i, j) = 1$, then i and j are adjacent i.e. $\mathbf{A}_{ij} = 1$, where $\mathbf{A}$ is the adjacency matrix.

Definition 2: Let $\mathcal{Z}(i)$ and $\mathcal{Z}(j)$ be the unique integer learnt for node i and j, then $f(\mathcal{Z}(i), \mathcal{Z}(j)) \approx \text{dist}(i, j) \leq l$, where $f(.)$ is some distance metric. We call this method of learning a unique integer for each node: *Numbering As a Position Encoding* (NAPE).

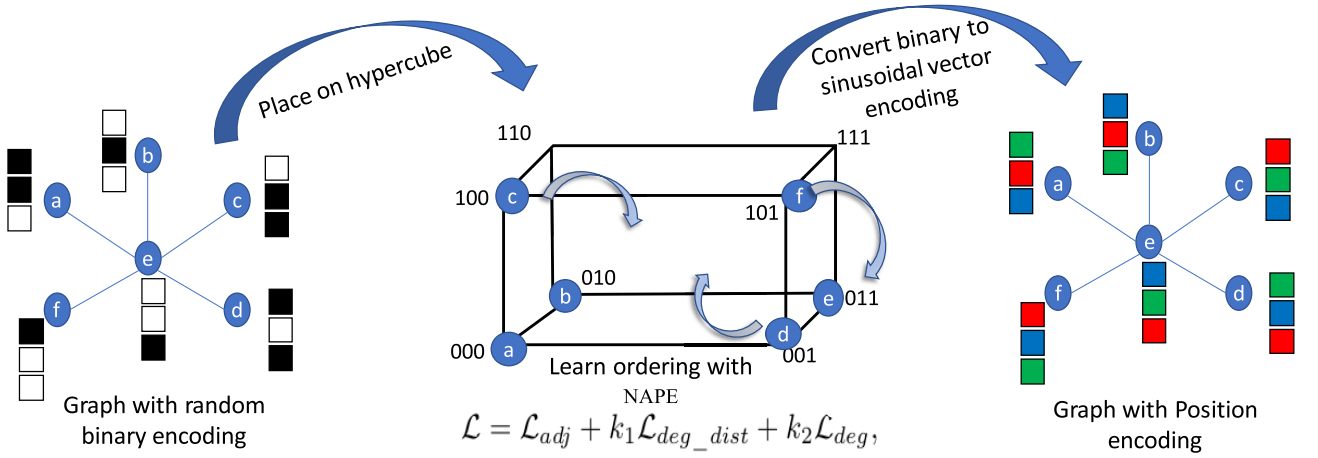


FIGURE 2. An illustration of the NAPE algorithm, where randomly initialized binary vectors for each node are reshuffled on the hypercube by minimizing the objective function $\mathcal{L}$. The distance on the hypercube approximates the distance on the graph, hence an error δ is introduced–which is bounded according to Def. 2. The optimal binary vectors are converted to sinusoidal vectors using (6).

1) SINUSOIDAL VECTORS WITH UNORDERED GRAPHS

The application of sinusoidal vector representations, as employed in [1], presents a challenge for unordered graphs due to the inherent reliance on node order. To establish a structured representation, we propose assigning positions to nodes based on their respective clusters within the graph. By organizing nodes within clusters and requiring an integer-based ordering, we propose the use of binary numbers to encode node positions. This approach facilitates distance computations within clusters and allows seamless conversion between binary and decimal representations.

Given a cube, each of the 8 vertices can be labelled using binary numbers 000 to 111, where numbers with a leading 0 correspond to base vertices and those with a leading 1 correspond to top vertices. For instance, in the graph $\mathcal{G}(V, E)$ depicted in Fig.2, nodes can be ordered by mapping them onto a labelled cube and learning a mapping such that their distance within clusters is approximated by their distance apart on the cube. The distance between each node on the cube can be computed using the Hamming distance metric $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{m} |x_i - y_i|$, where $\mathbf{x}$ and $\mathbf{y}$ represent the binary numbers assigned to each node, x_i and y_i are binary digits, m is the size of the bit we are working with (for our current graph in Fig.2, $m = 3$), and for a graph with N nodes, m has to be at least $\lceil \log_2 N \rceil$.

2) LEARNING THE NUMBERING SYSTEM

We present an algorithm designed to learn node positions within a hypercube (for sufficiently large N necessitating $m > 3$ bits). This algorithm aims to achieve two key objectives:

1) Proximity preservation: Nodes connected by edges in the original graph should be positioned close together within the hypercube embedding.
2) Degree-based locality: Nodes with identical degrees in the original graph should be mapped to nearby locations within the hypercube.

The rationale behind this second objective lies in the inherent relationship between a node's location and its functional role within the graph. For instance, central nodes with high degree act as communication hubs, while peripheral nodes with degree 1 represent endpoints or origins of signals. Isolated nodes with degree 0 are completely disconnected from the network. By enforcing degree-based locality, we introduce a structural constraint on the learnt embedding, guiding it towards a more meaningful representation. While this approach may not guarantee a unique solution, it demonstrably surpasses the inherent randomness of the $N!$ possible orderings.

Furthermore, incorporating degree information into the learnt position encoding facilitates the subsequent aggregation of nodes with similar degrees. To achieve this, we would learn to predict the degree of each node given the binary representation.

The task then is to learn a vector $\mathbf{P}_i$ for each node such that

$$\mathbf{Z}_i = \sigma(\gamma \mathbf{P}_i + \epsilon) \tag{1}$$

is the binary encoding for node i, γ and ϵ are learnable parameters, and $\sigma(.)$ is the sigmoid function. This would leave us with a position encoding matrix $\mathbf{Z} \in \{0, 1\}^{N \times m}$.

Given the adjacency matrix $\mathbf{A} \in \{0, 1\}^{N \times N}$ and the degree vector $\mathbf{d} \in \mathbb{R}^N$ of the corresponding adjacency matrix, we learn the optimal binary representations $\mathbf{Z}$ by clustering nodes based on the adjacency matrix (see (2)) and the degree vector (see (3)). This follows the rationale of proximity preservation and degree-based locality earlier explained.

$$\mathcal{L}_{adj} := \sum_{i \in V} -\frac{1}{|\mathcal{N}(i)|} \sum_{\rho \in \mathcal{N}(i)} \log \frac{\exp\left(d(\mathbf{Z}_i, \mathbf{Z}_\rho)\right)}{\sum_{a \in \mu(i)} \exp\left(d(\mathbf{Z}_i, \mathbf{Z}_a)\right)}, \tag{2}$$

$$\mathcal{L}_{deg_dist} :=$$

$$\sum_{i \in V} -\frac{1}{|\deg(i)|} \sum_{\rho \in \deg(i)} \log \frac{\exp\left(d(\mathbf{Z}_i, \mathbf{Z}_\rho)\right)}{\sum_{a \in \kappa(i)} \exp\left(d(\mathbf{Z}_i, \mathbf{Z}_a)\right)}, \quad (3)$$

where $\mathcal{N}(i)$ is the set containing all nodes adjacent to i, i.e., $\mathbf{A}_{ij} = 1$, and $\mu(i) = V - (\mathcal{N}(i) \cup \{i\})$ is the set of all non-neighbours of i, and $d(.)$ is the Hamming distance. In (3), $\deg(i)$ is the set containing all nodes that have the same degree as i, and $\kappa(i) = V - (\deg(i) \cup \{i\})$. Equation (2) and (3) is a variation of the equation proposed for clustering embeddings of the same class in [21], which itself is a variation of the contrastive learning technique [19], [22]. We have repurposed this equation to learn the binary encodings for nodes that are in the same local neighbourhood.

As earlier stated, to encode the degree of each node in the learnt position encoding we will learn to predict the degree of each node given the binary representation $\mathbf{Z}_i$. We do this by computing the L_2 norm (4) between the predicted degrees and $\mathbf{d}$.

$$\mathcal{L}_{deg} := ||\mathbf{Z}\mathbf{W}_{deg} - \mathbf{d}||_2, \quad (4)$$

where $\mathbf{W}_{deg} \in \mathbb{R}^m$ is a learnable weight vector used for predicting the degree of each node. Combining these results, we get the objective function (5) to learn the binary encoding vector for each node.

$$\mathcal{L} = \mathcal{L}_{adj} + \lambda_1 \mathcal{L}_{deg_dist} + \lambda_2 \mathcal{L}_{deg}, \quad (5)$$

where λ_1 and λ_2 are hyperparameters to be tuned. For graph networks where all nodes have a similar degree, for example, transportation networks [7], [24], we can relax the requirement for degree-based locality by fixing $\lambda_1 = \lambda_2 = 0$.

Algorithm 1 NAPE Algorithm

```
Input: A, d, λ1, λ2, epochs
Output: Z
N, deg ← get_neighbours(A)
μ, κ ← get_non-neighbours(A)
Z ← InitializeMatrix()
tmp ← ∞
for t = 1 to epochs do
    loss, Z ← Optimize(L(Z, d, N, deg, μ, κ, λ1, λ2))
    if(∀i, j ∈ Z : Z_i ≠ Z_j) and loss ≤ tmp
        Break
    end
    tmp ← loss
end
```

Adopting Sinusoidal Vectors. Using the computed optimal binary representation, $\mathbf{Z}$, we can get an approximate integer ordering for the graph nodes, and directly adopt the sinusoidal vector approach used in [1] to generate our position encodings. We first convert all learnt binary encodings $\mathbf{Z}_i$ to their decimal equivalent $t \in \mathbb{N}$, using the traditional place value numbering scheme. Now our position encoding takes the form,

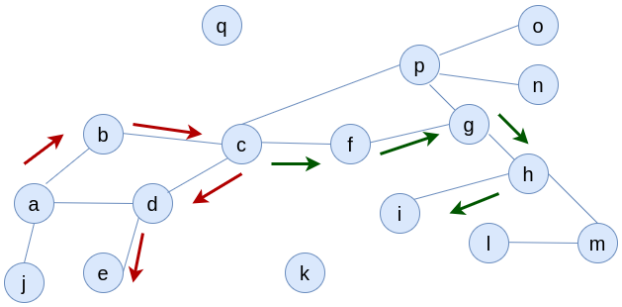


FIGURE 3. The figure describes random walks on a graph. The random walks are the red and green arrows; the red random walk starts at 'a' and terminates at 'e', while the green random walk starts at 'c' and terminates at 'i'. The nodes that co-occur in each random walk are recognised as the neighbours of the starting node.

$$\texttt{Pos}(t) := \begin{cases} \sin(\omega_b t), & \text{if } i = 2b \\ \cos(\omega_b t), & \text{if } i = 2b+1 \end{cases} \quad (6)$$

where,

$$\omega_b = \frac{1}{10000^{2b/d}} \quad (7)$$

$i = 1, 2, \ldots, d$ and $d = 2s$, $s \in \mathbb{N}$ is the length of the position encoding vector.

To utilize the position encoding in any GNN algorithm, we only need to add $\texttt{Pos}(t)$ to the transformed feature matrix of the graph dataset,

$$\mathbf{X}_i = \texttt{fea}(i)\mathbf{W}_d + \texttt{Pos}(\texttt{Dec}(\mathbf{Z}_i)). \quad (8)$$

$\mathbf{X}_i$ is the updated feature vector for node i, $\texttt{fea}(.) \in \mathbb{R}^{N \times f}$ is the feature matrix, $\mathbf{W}_d \in \mathbb{R}^{f \times d}$ is a learnable weight matrix to project the feature matrix to a dimension that matches the length of the position encoding, and $\texttt{Dec}(.)$ is the function that converts binary numbers to decimals.

IV. SCALING THE NAPE ALGORITHM

The NAPE algorithm in its current form is expensive to compute, and does not scale well to large graphs since the clustering equations (2) and (3) each have a computational cost of $\mathcal{O}(|V|^3)$. The NAPE algorithm brings no additional advantage if it shares the same $\mathcal{O}(|V|^3)$ cost as the eigenvector approach. We propose techniques to reduce the computational cost of the NAPE algorithm to $\mathcal{O}(|V|^2)$ by

- redefining what it means for a pair of nodes to be neighbours by taking advantage of node clusters; and
- we redefine the non-neighbours as negative samples from our node distribution.

A. NEIGHBOURS ON A RANDOM WALK

As earlier noted, the eigenvectors of a graph Laplacian define the clusters in a graph [5]. Each node belongs to some cluster which we can consider to be its local neighbourhood. If we simulate a random walk on the graph, under certain conditions, the node would mainly traverse nodes that are within its cluster. We thus redefine the neighbours of node i as those nodes that co-occur in a random walk of length l

starting at the node i. We see an instance of this in Fig.3 where the random walk starting at node a includes other nonadjacent nodes like c and e. We denote this set as $\mathcal{R}_l(i)$. For each node i, we perform m independent random walk, $\mathcal{R}_l^m(i)$. We therefore have that $\mathcal{N}(i) := \mathcal{R}_l^m(i)$.

We adopt the 2nd order random walk strategy proposed in [11], where the authors used the parameters p and q which forms a weighted probability vector, to alternate between a depth-first search (DFS) and a breadth-first search (BFS) on the graph. For our work, except otherwise stated, we always set $p < 1$ and $q > 1$, so we can achieve a BFS random walk, which follows from the clustering phenomenon we have earlier highlighted.

B. NON-NEIGHBOURS FROM NEGATIVE SAMPLING

In (2) we see the term

$$g(i,p) = \log \frac{\exp\left(d(\mathbf{Z}_i, \mathbf{Z}_\rho)\right)}{\sum_{a\in\mu(i)} \exp\left(d(\mathbf{Z}_i, \mathbf{Z}_a)\right)} \tag{9}$$

which is the main component of $\mathcal{L}_{adj}$. The number of non-neighbours $|\mu(i)|$ varies for each node, and in the worst case it is $|V|-1$. Following the recommendations in [23], we adopt the negative sampling approach to approximate (9), thereby reducing the number of non-neighbours from a worse case of $|V|-1$ to a constant size k. Here k is the number of negative samples randomly drawn from V. Hence, we have

$$\begin{aligned} g(i,p) \approx{}& \log\left(\sigma(d(\mathbf{Z}_i, \mathbf{Z}_p))\right) \\ & - \sum_{a=1}^{k} \log\left(\sigma(d(\mathbf{Z}_i, \mathbf{Z}_a))\right) \ , a \sim P_V \ . \end{aligned} \tag{10}$$

P_V is a random distribution over the graph nodes V, and is proportional to the degree of each node [11]. It should be noted that the authors of [23] gave their approximation using the dot product between the vector representations $\mathbf{Z}_i$. In our algorithm, this has been replaced with the Hamming distance $d(.)$ since we are computing the distances between binary vectors on the vertices of a hypercube.

C. DEGREE DISTRIBUTION

Given $\mathcal{G}(V,E)$, we derive a graph $\mathcal{H}(V,\tilde{E})$ from $\mathcal{G}(V,E)$ such that only nodes having the same degree in $\mathcal{G}(V,E)$ are connected in $\mathcal{H}(V,\tilde{E})$ Fig.4. We note that the graph $\mathcal{H}(V,\tilde{E})$ comprise disconnected clusters of complete subgraphs. This is particularly useful when the original graph $\mathcal{G}(V,E)$ has multiple isolated nodes, as this introduces similarities between these isolated nodes.

We have not naively assumed that the computational cost for (2) also applies for (3). Given a subgraph $S \subset \mathcal{H}(V,\tilde{E})$, s.t. $S := \{u, v$: each node has a degree of $0\}$. The number of non-neighbours for $u \in S$, i.e. nodes with degree at least 1, is $\kappa(u) = |V|-2$. Similarly, $\deg(i)$, the subset of nodes having the same degree as node i, is assumed to be large for a large graph $\mathcal{H}(V,\tilde{E})$.

To obtain a scaled-down set of $\deg(i) \in \mathcal{H}(V,\tilde{E})$, we randomly sample l nodes connected to i. Since every connected subgraph of $\mathcal{H}(V,\tilde{E})$ is a complete graph, a random walk will not yield any additional information on the local neighbours of a given node i.

Similarly, to scale down $\kappa(i) \in \mathcal{H}(V,\tilde{E})$, we randomly sampled k nodes not within the subgraph associated with node i. These randomly sampled k nodes would be used as negative-samples, just as we did in (10). We do not enforce the probability distribution for the k negative samples to be proportional to the degree distribution.

D. RESULTING COMPUTATIONAL COST

With the introduction of the random walk $\mathcal{R}_l^m(i)$ to approximate neighbours of a node i, and non-neighbours as k negative samples, the computational cost for the loss $\mathcal{L}$ is $\mathcal{O}(|V|^2)$ with the condition that $lmk \leq |V|$. We assume that the computational cost of $\mathcal{L}_{deg}$ is negligible in comparison to $\mathcal{L}_{adj}$ and $\mathcal{L}_{deg_dist}$.

E. FREQUENCY TUNING

We observed that the frequency ω_b in (6) affects how robust the generated position encodings are for any graph. In particular, the frequency is related to the number of nodes in the graph. This is made obvious when we consider the eigenvector for the Laplacian of a path graph with $|V|$ nodes, given as (6), with the condition that

$$\omega_b = \frac{2\pi b}{|V|}. \tag{11}$$

Equating (7) and (11) yields

$$\eta = Q.|V|^\beta, \tag{12}$$

where $Q = (2\pi b)^{-d/2b}$, $\beta = d/2b$, and η is the arbitrary constant 10000. Therefore,

$$\eta \propto |V|^\beta. \tag{13}$$

So, for the datasets we work with, we changed the default parameter in the frequency ω_b from 10000 to some other value η proportional to the number of nodes in the corresponding graph.

V. EXPERIMENT

We conducted experiments for node classification, link prediction on large-scale graph datasets, and action recognition using human skeletal graphs.

A. DATASETS

1) WIKICS DATASET

This dataset [25] consists of Wikipedia articles in computer science, divided into 10 branches representing the classes for node categorization. Nodes represent articles, and edges represent hyperlinks between them. Node features are the average of the GloVe word embeddings of the articles. The WikiCS dataset contains 11,701 nodes and 216,123 edges. The task is to classify a subset of nodes into their respective branches.

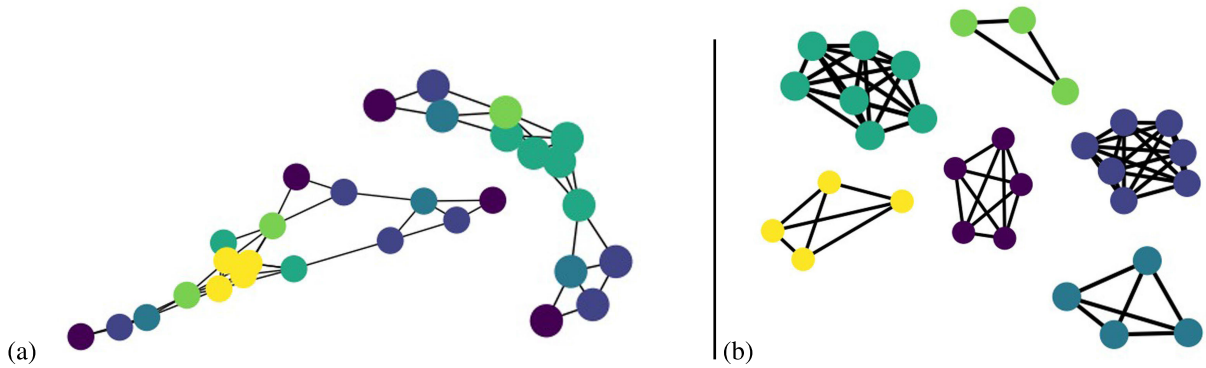


FIGURE 4. Graph (b) is derived by connecting nodes in (a) that share the same degree, resulting in a new graph denoted as $\mathcal{H}(V, \tilde{E})$ comprising complete subgraphs. To enhance visual clarity, nodes with identical degrees are assigned the same colour, facilitating the discernment of structural patterns within the graph.

2) PUBMED DATASET

This dataset [26] includes 19,717 scientific publications on diabetes, categorized into three classes. Nodes represent publications, and edges represent citation links, forming a directed network. Node features are TF-IDF-weighted word vectors, highlighting the relative importance of words within a publication's content. The task is node classification, predicting the class of new publications. According to [25] the distribution of the ratio of neighbours belonging to the same class, shows that the PubMed dataset has a higher proportion of neighbouring nodes with the same class (compared to the WikiCS dataset). So We expect the PubMed dataset to benefit most from the positional encoding because neighbours are better identified with the positional encoding.

3) OGBL-COLLAB DATASET

This dataset [27] is an undirected multi-graph representing author collaborations. Nodes are authors, with features as the average word embeddings of their publications. Edges represent co-authorship, with meta-data on the year and the number of co-authored papers. The dataset has 235,868 nodes and 2,358,104 edges. The task is link prediction, predicting future collaborations between authors.

4) NTU RGB+D

This dataset [28] includes over 56,000 samples from 40 subjects performing 60 actions, recorded by three cameras positioned at -45Â°, 0Â°, and +45Â°, at various heights and distances. Skeletal data is captured using a Kinect V2 camera, resulting in a graph with 25 nodes, each representing 3D coordinates (x, y, z) of body joints. The dataset is preprocessed as suggested in [18], and split into training and evaluation sets using cross-subject and cross-view splits.

B. PARAMETER SETTINGS

1) SCALED-NAPE FOR GNN

For all graph datasets, we used node2vec [11] for random walks to approximate neighbors. We set $p = 0.25$ and $q = 2.00$ for a breadth-first type random walk. In the ablation studies section, we considered the effectiveness of the depth-first random walk by setting $p = 2.00$ and $q = 0.25$.

For the WikiCS dataset, we performed $m = 1$ random walk of length $l = 100$, with 10 negative samples ($k = 10$). Hyperparameters for $\mathcal{L}$ were $\lambda_1 = 10^{-4}$ and $\lambda_2 = 1$. Binary encoding dimension was $d = 22$, and $\eta = 1 \times 10^5$.

For the PubMed dataset, we performed $m = 1$ random walk of length $l = 80$, with 15 negative samples ($k = 15$). Hyperparameters were $\lambda_1 = 10^{-3}$ and $\lambda_2 = 1$. Binary encoding dimension was $d = 24$, and $\eta = 1.5 \times 10^5$.

For the OGBL-collab dataset, we performed $m = 2$ independent random walks of length $l = 180$, with 20 negative samples ($k = 20$). Binary encoding dimension was $d = 30$, and $\eta = 5 \times 10^6$. All other parameters matched those for the WikiCS setup.

The scaled-NAPE model was optimized using an Intel Core i5-11500 CPU (6 cores/12 threads) with 64GB RAM. Alongside this, all the GNN models were trained using an Nvidia A5000 GPU (24GB). WikiCS and OGBL-collab datasets were trained for a maximum of 10 epochs with the ADAM optimizer, while the PubMed dataset was trained for 5 epochs, all with a learning rate of 10^{-2}.

2) NAPE FOR GNN

We learned position encodings for the WikiCS and PubMed datasets using the original NAPE algorithm. All parameters were consistent except for η. For the WikiCS dataset, $\eta = 5 \times 10^4$. For the PubMed dataset, with 60% and 70% of non-neighbors, η was 5×10^5 and 5×10^4, respectively.

3) NAPE FOR ACTION RECOGNITION

For action recognition models, we set $\lambda_1 = \lambda_2 = 1$ and $d = 8$. The NAPE model was optimized using the ADAM optimizer with a learning rate of 0.1 for 400 epochs.

4) NODE EMBEDDING-BASED POSITIONAL ENCODING

We used the node2vec algorithm [11] to embed the nodes of all graphs we have reported. We used the same vector length d in the NAPE position encoding for the node embeddings

for each graph dataset. This is to ensure a fair comparison of these techniques.

5) DISTANCE-BASED POSITIONAL ENCODING

Following the prescription in [6], $\frac{1}{A^h + 1}$ yields a distance-based positional encoding, where $\mathbf{A}$ is the adjacency matrix and h is the number of hops. For all datasets, we set $h = 15$.

6) LEARNABLE POSITIONAL ENCODING

We include a learnable matrix in all the GNN models reported. The entries of the matrix are updated during training, and this would serve as the learnt positional encoding for each graph and its corresponding GNN model.

7) GRAPH NEURAL NETWORKS (GNN)

For a fair comparison, we used the same parameter settings as in the Benchmarking Graph Neural Networks paper [2]. We ran multiple realizations and reported the mean metric with standard deviation. Our NAPE position encoding results were compared with the eigenvector approach in [2].

8) ACTION RECOGNITION MODELS

Across all action recognition models we experimented with, we used the same parameter settings provided in the original work. The same position encoding obtained using the default NAPE algorithm was added to all the action recognition models.

VI. RESULTS

A. LARGE-SCALE GRAPH DATASETS

Table 1 demonstrates that our NAPE position encoding enhances the accuracy of the MoNET algorithm for the WikiCS dataset, surpassing the performance of all other position encoding approaches, except the learnable position encoding. The learnable position encoding introduces about 10^6 new parameters to the model, thereby increasing training time and memory requirements compared to other approaches. On the other hand, the Node-based position encoding and the Laplacian eigenvector position encoding have large deviations, which suggests their lack of robustness to varied initial conditions in the GNN model. Although the distance-based position encoding appears to be the fastest for the WikiCS dataset, the performance is abysmal. The scaled-NAPE algorithm on the other hand is just behind the Node-based position encoding, but significantly faster than the Laplacian eigenvector approach, requiring about half the time (including the time to simulate the random walks). This efficiency is expected, as the scaled NAPE algorithm achieves a computational complexity of $\mathcal{O}(|V|^2)$ when $lmk \leq |V|$. The default NAPE algorithm yields a higher accuracy but requires three times the computation time compared to the scaled NAPE algorithm.

For the PubMed dataset, Table 1 shows that NAPE improves the baseline MoNET model. Although the

TABLE 1. Performance comparison of GNN models using node-based position encoding, distance-based position encoding, Laplacian eigenvectors and NAPE for position encodings on the WikiCS, PubMed, and OGBL-collab datasets. The reported time is the duration to compute each of the position encodings.

WikiCS		
Algorithm	Test Acc + s.d. (%)	Time
MoNET [29]	77.564 ± 0.772	N/A
MoNET + Node Embed[†][11]	77.528 ± 0.946	1.16 mins[1]
MoNET + Dist Enc[†]	24.730 ± 2.314	**1.07** mins
MoNET + Learn Enc	$\mathbf{78.337 \pm 0.688}$	N/A
MoNET + Lap Eig[†][2]	77.342 ± 1.000	5.25 mins
MoNET + scaled-NAPE[†]	77.740 ± 0.595	2.50 mins[1]
MoNET + NAPE[†]	77.772 ± 0.653	7.00 mins
PubMed		
Algorithm	Test Acc + s.d. (%)	Time
MoNET [29]	82.948 ± 0.232	N/A
MoNET + Node Embed[11]	$\mathbf{89.472 \pm 0.459}$	**0.13** mins[1]
MoNET + Dist Enc	84.098 ± 0.500	4.95 mins
MoNET + Learn Enc	$81.939 \pm 0.0.683$	N/A
MoNET + Lap Eig[†][2]	85.342 ± 0.462	22.00 mins
MoNET + scaled-NAPE	84.460 ± 0.442	1.60 mins[1]
MoNET + NAPE (neg–0.7)[2]	83.596 ± 0.353	62.90 mins
MoNET + NAPE (neg–0.6)[3]	83.214 ± 0.533	54.40 mins
OGBL-collab		
Algorithm	Test@50 Hits (%)	Time
GatedGCN [30]	53.900 ± 0.596	N/A
GatedGCN + Node Embed[11]	$\mathbf{54.647 \pm 0.632}$	**8.64** mins[1]
GatedGCN + Dist Enc	OOM	-
GatedGCN + Learn Enc	49.978 ± 2.232	N/A
GatedGCN + Lap Eig [2]	OOM	-
GatedGCN + scaled-NAPE	50.410 ± 1.869	766.80 mins[1]

[†] : Alternated the sign of position encoding during training.
[1] This includes the time to compute the random walks.
[2] NAPE used a random sample of 70% of non-adjacent nodes as non-neighbours for each node.
[3] NAPE used a random sample of 60% of non-adjacent nodes as non-neighbours for each node.
OOM: Out of Memory

Node-based position encoding achieves the highest accuracy, followed by the Laplacian eigenvectors, our scaled-NAPE position encoding can be computed in about 7% of the time needed for the Laplacian eigenvectors. However, regarding computational time, the scaled-NAPE algorithm is second to the Node-based position encoding. Given our computational constraints, training the default NAPE algorithm for the PubMed dataset would exhaust available memory. The non-adjacent nodes (non-neighbours) $\mu(i)$ for each node i consume a substantial portion of memory during training. We addressed this by randomly sampling 60% and 70% of the non-neighbors for training. Although this approach marginally improves the baseline model, it does not match the accuracy or compute time of the scaled-NAPE position encoding, highlighting the importance of the negative sampling technique for scaling the default NAPE algorithm. The accuracy of the learnable position encoding drops below the baseline, even after introducing more than

TABLE 2. Accuracy of action recognition models with and without NAPE position encoding.

Algorithm	Accuracy			
	Cross-View (%)		Cross-Subject (%)	
	Baseline	NAPE	Baseline	NAPE
STGCN [31]	92.72	**93.66**	85.05	**85.70**
2s-AGCN [32]	94.24	**94.74**	87.43	**87.84**
Info-GCN [18]	95.45	**95.48**	90.55	**90.57**

1.5×10^6 training parameters. The distance-based encoding improves the baseline but is behind the scaled-NAPE and requires almost 4 times the compute time than the scaled-NAPE.

For the OGBL-collab dataset, the Node-based position encoding outperforms other techniques both in accuracy and compute time. We were unable to report the performance of the GatedGCN model using the Laplacian eigenvectors and the distance-based position encoding due to the dataset's large size (235,868 nodes). The larger $|V| \times |V|$ Laplacian matrix results in a higher computational cost $\mathcal{O}(|V|^3)$, leading to memory exhaustion–same applies to the distance-based position encoding. Additionally, both NAPE and the learnable position encodings are below the baseline by approximately 3%. The high variance in hits indicates the sensitivity of both approaches to initial conditions on this dataset. We attribute these drawbacks, for NAPE, to the OGBL-collab dataset being a multi-graph with additional edge features that influence node embeddings. Unlike the WikiCS and PubMed datasets which are simple graphs, our NAPE position encoding algorithm fails to capture the clustering information in a multi-graph, therefore making the learnt position encoding ineffectual for the link prediction tasks. A detailed future research is needed in designing the NAPE algorithm for multi-graphs.

It should be noted that the implementation of the node2vec [11] algorithm, which is used as the Node-based position encoding, has been highly optimized. Under similar optimization gains, we expect that the NAPE algorithm can match the compute time for node2vec.

Also, the abysmal performance of the distance-based position encoding observed for the WikiCS dataset is potentially due to the number of hops h used in computing the position encoding. Although increasing h can in theory improve the accuracy, the constraints of the computational requirement make it infeasible.

B. ACTION RECOGNITION DATASET

Figure 1 shows the learnt ordering for the nodes of the human skeletal graph using NAPE. Using the sinusoidal vector equation in (6) and the learnt ordering, we generated the position encoding for the skeletal graph, and applied it to all action recognition models reported.

Table 2 shows that the NAPE position encoding enhances the accuracy of all action recognition models. Except for the Info-GCN model [18], which already includes a position encoding in its baseline, the other models do not incorporate position encodings. Replacing the learnable position encoding parameters in Info-GCN with NAPE matches the model's performance. In the Info-GCN model, the learnable position encoding functions as a learnable bias parameter in a linear transformation, useful only within the model. In contrast, the NAPE position encoding is applicable across various models involving the skeletal graph for which it was designed. Similar accuracy improvements are observed for STGCN [31] and 2s-AGCN [32], which is expected as 2s-AGCN directly builds on STGCN. Similar improvements would be expected for other models built on STGCN, such as AS-GCN [33] and ST-TR-AGCN [34].

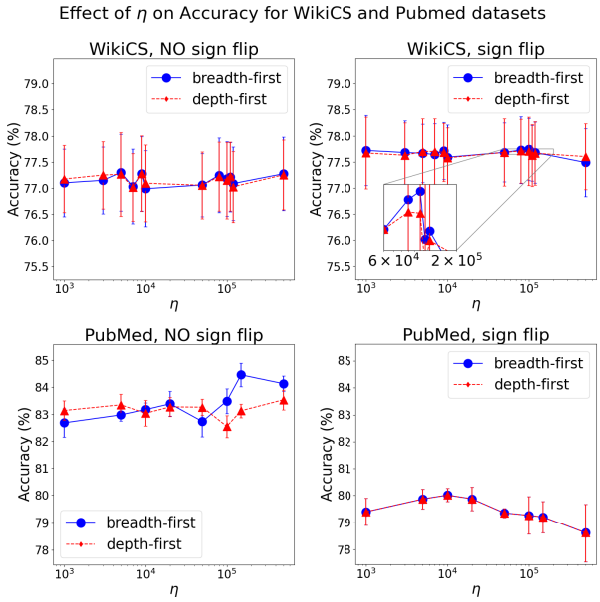


FIGURE 5. Variations in GNN algorithm accuracy for different values of η for the WikiCS and PubMed datasets. The plots on the right correspond to results where the signs of the position encodings were randomly flipped during training. Each figure shows the variation in the model accuracy using NAPE position encoding learnt with a depth-first (red) and breadth-first (blue) random walk.

As a point of note, the disparity between the cross-view and cross-subject accuracy is due to the inherent difficulty of distinguishing the same action performed by different subjects. We expect that there will be subtleties in each subject's approach to performing an action. This difficulty due to the subtleties is reflected in the reported cross-subject accuracy, which is observed across the literature. An additional factor that enhances the classification accuracy for the cross-view split is the preprocessing step prior to passing the data to the model. The preprocessing step in SGN [35], which we followed, normalized the coordinate data of each skeleton such that the centroid is at the origin, and each skeleton faces the same direction.

C. ABLATION STUDIES

1) EFFECT OF η AND RANDOM SIGN FLIPPLING ON THE ROBUSTNESS OF NAPE

We showed in (12) that η is proportional to $|V|^\beta$, and the choice of η influences the robustness of the generated

position encoding. Figure 5 shows how model accuracy is significantly influenced by the choice of η, especially for the PubMed dataset. Randomly flipping the signs of the position encodings during training [2] is useful only for an undirected (simple) graph, like the WikiCS dataset. The alternating signs indicate no order exists between adjacent nodes on the graph. This is why we see a significant drop in the accuracy of the PubMed dataset, which is a directed (simple) graph when the sign of the learnt NAPE position encoding was randomly flipped during training.

2) BREADTH-FIRST VS DEPTH-FIRST

We earlier explained that adopting a breadth-first random walk for scaling our NAPE algorithm would approximate the clustering properties observed in the Laplacian eigenvector. To support this, fig. 5 compares the model accuracy for NAPE position encoding learnt using breadth-first and depth-first random walk. For both the WikiCS and the PubMed datasets (with and without sign flip respectively), the breadth-first random walk results significantly outperform the depth-first result, particularly in the regions of η we expect NAPE to be most effective.

VII. CONCLUSION

We have introduced *NAPE*, a novel approach for generating position encodings for nodes in a graph. Our algorithm addresses the challenge of node ordering in a graph, eliminating the restrictive $|V|!$ possible orderings. We defined and implemented an ordering that encapsulates both topological and relational information between nodes. Additionally, we demonstrated the scalability of the NAPE algorithm for large-scale graphs using simulated random walks and negative sampling techniques.

As a direct comparison to Laplacian eigenvectors, our proposed method compares favourably on benchmark graph datasets although with some limitations. Unlike the position encoding method in [1], which provides unique encodings for each node in a path graph, our graph position encodings are not always unique, potentially compromising unique node embeddings. However, in the datasets we evaluated, less than 0.5% of nodes shared the same position encoding.

Also, our NAPE algorithm requires a heuristic search for suitable hyperparameters to give the *best* position encoding, unlike the Laplacian eigenvectors or the distance-based position encoding that require no fine-tuning. In addition, the precomputed random walks can add more overhead to the compute time as the number of hops increases. In practice, this is easily resolved using parallelization. Unlike the distance-based positional encoding, increasing the number of hops does not suffer both memory and compute resources.

Unlike node2vec [11] where the vectors exist in $\mathbb{R}^d$, the choice of binary vectors $\{0, 1\}^d$ for the NAPE algorithm, imposes a limitation on the available volume for nodes within a cluster. So for the same number of nodes, we will need a sufficiently larger value of d for NAPE compared to node2vec to obtain a reasonable embedding for clusters.

While the Laplacian eigenvector approach ensures non-unique position encodings due to the algebraic multiplicity of eigenvalues, the NAPE algorithm theoretically offers unique position encodings, though not guaranteed to be optimal. We have shown that the sign invariance criteria is related to the direction of the edges in the (simple) graph. This makes sign flips during training unnecessary for directed (simple) graphs and necessary for undirected (simple) graphs.

Moreover, our approach is currently limited to datasets containing a single graph, which emphasizes its utility for large-scale graphs. Our results indicate that the NAPE algorithm reliably improves the accuracy of the baseline across different datasets, with the computational requirements scaling favourably with larger graphs. Although the effectiveness of the NAPE position encoding is evident for simple graphs like the WikiCS and PubMed datasets, further research is required to modify the NAPE algorithm for multi-graphs, as suggested by results from the OGBL-collab dataset. In addition, our current work focuses on non-weighted graphs with binary edges. More research is required to modify the current NAPE algorithm for a weighted graph, where the importance of edges would affect the importance of neighbouring nodes and change the definition of a local cluster.

We have also demonstrated that the NAPE position encoding learnt for one human skeletal graph can be applied across different action recognition models. While the relatively small size of the human skeletal graph may not justify the computational effort to learn a separate position encoding for an action recognition model, the NAPE position encoding provides a versatile solution. This ''one size fits all'' benefit is particularly advantageous for training action recognition models with large parameter sizes, reducing the need for model-specific position encodings, as seen in [18]. Also learning a position encoding by including it in the model parameters, increases our model size significantly, and the results of Table 1 suggest that learnable position encodings do not reliably improve a model's accuracy, especially with increasing graph size.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, ''Attention is all you need,'' in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[2] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, ''Benchmarking graph neural networks,'' 2020, *arXiv:2003.00982*.

[3] S. Geisler, Y. Li, D. J. Mankowitz, A. T. Cemgil, S. Günnemann, and C. Paduraru, ''Transformers meet directed graphs,'' in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 11144–11172.

[4] D. Lim, J. Robinson, L. Zhao, T. Smidt, S. Sra, H. Maron, and S. Jegelka, ''Sign and basis invariant networks for spectral graph representation learning,'' 2022, *arXiv:2202.13013*.

[5] M. Fiedler, ''Laplacian of graphs and algebraic connectivity,'' *Banach Center Publications*, vol. 25, no. 1, pp. 57–70, 1989.

[6] J. You, R. Ying, and J. Leskovec, ''Position-aware graph neural networks,'' in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7134–7143.

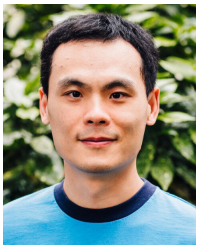
[7] K. Klemmer, N. S. Safir, and D. B. Neill, ''Positional encoder graph neural networks for geographic data,'' in *Proc. Int. Conf. Artif. Intell. Statist.*, 2023, pp. 1379–1389.

[8] R. Brüel-Gabrielsson, M. Yurochkin, and J. Solomon, "Rewiring with positional encodings for graph neural networks," 2022, *arXiv:2201.12674*.
[9] Y. Huang, W. Lu, J. Robinson, Y. Yang, M. Zhang, S. Jegelka, and P. Li, "On the stability of expressive positional encodings for graphs," 2023, *arXiv:2310.02579*.
[10] B. Xu, H. Shen, Q. Cao, Y. Qiu, and X. Cheng, "Graph wavelet neural network," 2019, *arXiv:1904.07785*.
[11] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 855–864.
[12] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 701–710.
[13] W. Park, W. Chang, D. Lee, J. Kim, and S.-W. Hwang, "GRPE: Relative positional encoding for graph transformer," 2022, *arXiv:2201.12787*.
[14] P. Li, Y. Wang, H. Wang, and J. Leskovec, "Distance encoding: Design provably more powerful neural networks for graph representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 4465–4478.
[15] R. B. Gabrielsson, M. Yurochkin, and J. Solomon, "Rewiring with positional encodings for graph neural networks," *Trans. Mach. Learn. Res.*, 2023.
[16] V. Prakash Dwivedi, A. Tuan Luu, T. Laurent, Y. Bengio, and X. Bresson, "Graph neural networks with learnable structural and positional representations," 2021, *arXiv:2110.07875*.
[17] Y. He, C. Bodnar, and P. Lio, "Sheaf-based positional encodings for graph neural networks," in *Proc. NeurIPS*, 2023, pp. 1–18.
[18] H.-G. Chi, M. H. Ha, S. Chi, S. W. Lee, Q. Huang, and K. Ramani, "InfoGCN: Representation learning for human skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 20154–20164.
[19] O. Henaff, "Data-efficient image recognition with contrastive predictive coding," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2020, pp. 4182–4192.
[20] P. Veličković, W. Fedus, W. L. Hamilton, P. Lio, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," 2018, *arXiv:1809.10341*.
[21] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 18661–18673.
[22] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *Technologies*, vol. 9, no. 1, p. 2, Dec. 2020.
[23] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 1–9.
[24] T. Liu and H. Meidani, "End-to-end heterogeneous graph neural networks for traffic assignment," 2023, *arXiv:2310.13193*.
[25] P. Mernyei and C. Cangea, "Wiki-CS: A wikipedia-based benchmark for graph neural networks," 2020, *arXiv:2007.02901*.
[26] Z. Yang, W. Cohen, and R. Salakhudinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 40–48.
[27] K. Wang, Z. Shen, C. Huang, C.-H. Wu, Y. Dong, and A. Kanakia, "Microsoft academic graph: When experts are not enough," *Quant. Sci. Stud.*, vol. 1, no. 1, pp. 396–413, Feb. 2020.
[28] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "NTU RGB+D: A large scale dataset for 3D human activity analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1010–1019.
[29] F. Monti, D. Boscaini, J. Masci, E. Rodola, J. Svoboda, and M. M. Bronstein, "Geometric deep learning on graphs and manifolds using mixture model CNNs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5425–5434.
[30] X. Bresson and T. Laurent, "Residual gated graph ConvNets," 2017, *arXiv:1711.07553*.
[31] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 1–9.
[32] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 12018–12027.
[33] M. Li, S. Chen, X. Chen, Y. Zhang, Y. Wang, and Q. Tian, "Actional-structural graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3590–3598.
[34] C. Plizzari, M. Cannici, and M. Matteucci, "Skeleton-based action recognition via spatial and temporal transformer networks," *Comput. Vis. Image Understand.*, vol. 208, Jul. 2021, Art. no. 103219.
[35] P. Zhang, C. Lan, W. Zeng, J. Xing, J. Xue, and N. Zheng, "Semantics-guided neural networks for efficient skeleton-based human action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1109–1118.



OLAYINKA AJAYI received the B.Sc. degree in mathematics and computer science from the University of Port Harcourt, the M.Sc. degree in applied mathematical science with climate change impact modeling from Heriot-Watt University, and the M.Sc. degree in mathematics of systems from the University of Warwick, where he is currently pursuing the Ph.D. degree, where he is part of the EPSRC and MRC Centre for Ph.D. Training in mathematics for real-world systems.

His research interests span cryptography, fluid mechanics, and machine learning, with his current focus on developing deep spatiotemporal models for video representation learning. Notable projects include his work on the RivestShamirAdleman (RSA) Public Key Cryptosystem and the Maxey-Riley equation for particle trajectories in fluid dynamics, reflecting his broad academic interests and innovative approach to problem-solving. His Ph.D. Research, supervised by Prof. Hongkai Wen and Dr. Tanaya Guha, involves designing new position encoding algorithms for large-scale graphs and enhancing human action recognition in video datasets. He has presented his findings at various academic conferences and seminars, contributing to the field of computer vision and machine learning. Additionally, he has served as a Teaching Assistant and has been actively involved in organizing academic events. His commitment to academic excellence and interdisciplinary research is evident in his extensive body of work, which aims to address real-world challenges through advanced mathematical and computational techniques.



HONGKAI WEN received the D.Phil. degree in computer science from the University of Oxford, in 2014. He was a Postdoctoral Researcher with Oxford Computer Science and Robotics Institute. He is currently a Professor with the Department of Computer Science, University of Warwick. He holds a joint appointment as a Senior Research Scientist with Samsung AI Center, Cambridge, and is a fellow of the Alan Turing Institute. His research interests include scalable machine learning systems, on-device AI, and pervasive data science.



TANAYA GUHA received the Ph.D. degree in electrical and computer engineering from the University of British Columbia (UBC), Vancouver, Canada. She is currently a Senior Lecturer (Associate Professor) with the School of Computing Science, University of Glasgow. Her research is focused on developing AI capabilities to understand human behavior combining deep learning, computer vision, and signal/speech Processing. She is a Program Committee Chair for BMVC 2024. She serves on the Editorial Board of IEEE Pervasive Computing and *APSIPA Transactions on Signal and Information Processing*, and was an Editorial Board Member of *Scientific Reports* earlier.